

Entrega Final

- APAAD -

Marcelo Ferreiro Sánchez
Pablo Chantada Saborido
Gillermo García Engelmo
Pepe Romero Conde

11 de noviembre de 2025

Índice

1. Introducción	2
2. Diseño del modelo lógico e implementación del DW en Oracle	2
2.1. Rediseño do modelo da práctica anterior	2
2.2. Toma de decisións	2
2.3. Creación das Táboas	4
2.3.1. Táboas de dimensión	4
2.3.2. Táboas de dimensión que cambian lentamente, implementadas con SCD tipo 2	5
2.3.3. Táboas ponte	6
2.3.4. Táboa de feitos	6
3. Diseño e implementación del ETL	7
4. Explotación del DW	8
Consulta 1	8
Consulta 2	9
Consulta 3	10

1. Introducción

blablabla

2. Diseño del modelo lógico e implementación del DW en Oracle

- Proponer el diagrama del modelo lógico para el diseño realizado (normalmente será un modelo sencillo, en estrella o en copo de nieve), incluyendo una breve explicación de las decisiones tomadas y crear las tablas resultantes en Oracle.
- Crear las tablas necesarias en Oracle.

2.1. Rediseño do modelo da práctica anterior

Despóis da corrección da entrega anterior modificamos o DFM do seguinte xeito:

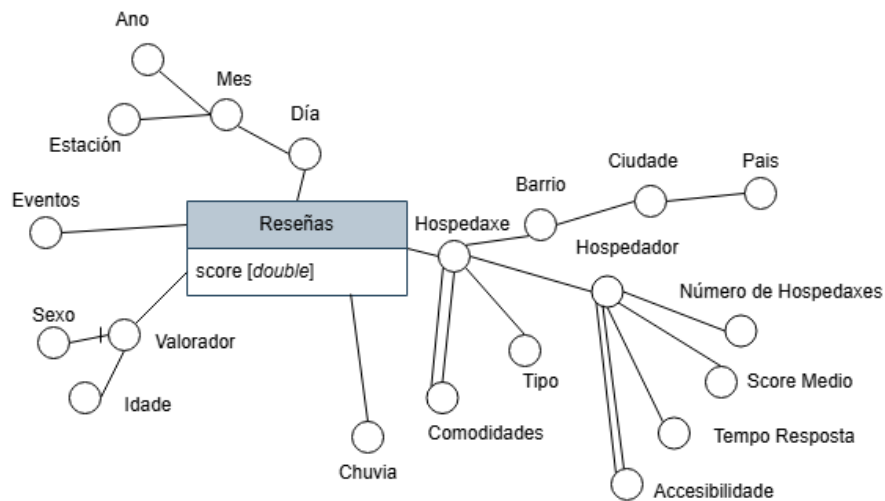
- Baixamos as dimensións de **Hospedador** e **Lugar** ó nivel de **Hospedaxe** para evitar o modelo *Copo de neve*, o cal atopamos vantaxoso por gañar simpleza do modelo e velocidade das consultas (ver [1]).
- Actualizamos o DFM cos atributos da dimensión **Eventos** especificados no informe.
- Cambiamos as SCD a tipo 2 para conservar os datos do pasado e poder analízalos (dimensións **Hospedador** e **Valorador**).
- Cambios sobre o resto de atributos para busca coherencia co informe anterior.

Estos cambios pódense ver reflectidos na Figura 1. Na seguinte subsección veremos como implementar estes cambios.

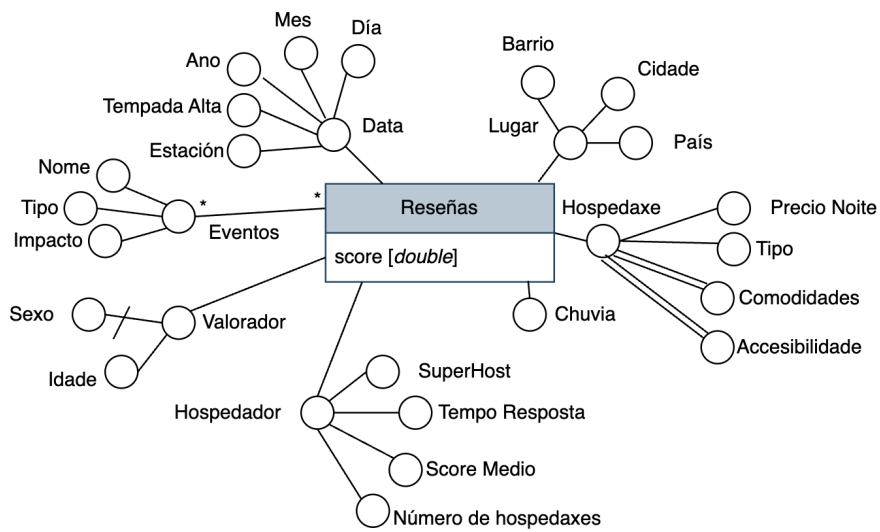
2.2. Toma de decisións

Con todo resulta un modelo en estrela. Onde:

- A dimensión **Tempo** está denormalizada, para que en vez de estar representada de forma xerárquica, sea tipo Estrela. Ademáis, como proposto nas teóricas usamos o atributo **data** como chave primaria (identifica o que ten que identificar).
- Á dimensión **Lugar** tivemos que crearlle unha chave primaria *surrogada* pois pode haber, por exemplo, máis dun barrio *chinés*. Con todo, si forzamos a non duplicidade das ternas (**barrio**, **cidade**, **pais**) con UNIQUE KEY.
- A dimensión **Hospedaxe** ten dos atributos multievaluados (**Comodidades** e **Accesibilidade**, que describen prestacións do aloxamento e formas de chegar a él, respectivamente) e modelárasen con Táboas Ponte.



(a) DFM Orixinal



(b) DFM Modificado

Figura 1: Diagrama do Modelo de Feitos Dimensional (*DFM*) antes e despois da corrección

- A dimensión **Eventos** usa claves foráneas para asegurar coherencia e restricións para asegurar que os valores categóricos son os esperados e para evitar duplicados. Ademáis inclúe un índice (**INDEX**) para que as consultas do tipo:

```

1 SELECT e.evento_id, e.evento_nome, e.impacto
2 FROM dim_eventos e
3 JOIN dim_lugar l ON e.lugar_id = l.lugar_id
4 WHERE e.evento_data = DATE '2024-11-10'
5 AND l.cidade = 'Madrid';

```

sexan veloces, e xustamente son ese tipo de consultas ás que nos interesan (os eventos ocorren nun momento nunha cidade) (ver [2]).

- Os atributos **comodidade** e **accesibilidade** ó seren multievaluados teñen que ser modelados cunha táboa ponte. Por tanto merecen a súa dimensión propia (ademáis da ponte).
- A dimensión **Chuvia** é dexenerada pois a única métrica que aporta son os mm de precipitación. Polo tanto ben a podemos incluír na táboa de feitos.

2.3. Creación das Táboas

A continuación mostrase as instrucións SQL para crealas táboas. Agrupadas segundo o tipo.

2.3.1. Táboas de dimensión

```

1 CREATE TABLE dim_tempo (
2     data DATE PRIMARY KEY,
3     dia NUMBER NOT NULL,
4     mes NUMBER NOT NULL,
5     ano NUMBER NOT NULL,
6     estacion VARCHAR2(10) NOT NULL,
7     tempada_alta NUMBER(1) NOT NULL -- 1=TRUE, 0=FALSE
8 );

```

```

1 CREATE TABLE dim_lugar (
2     lugar_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
3     barrio VARCHAR2(100) NOT NULL,
4     cidade VARCHAR2(100) NOT NULL,
5     pais VARCHAR2(100) NOT NULL,
6     CONSTRAINT uk_lugar UNIQUE (barrio, cidade, pais)
7 );

```

```

1 CREATE TABLE dim_hospedaxe (
2     hospedaxe_id NUMBER PRIMARY KEY, -- Xa a ti amos na OLTP
3     precio_noite NUMBER(10,2) NOT NULL,
4     tipo VARCHAR2(50) NOT NULL
5 );

```

```

1 CREATE TABLE dim_eventos (
2     evento_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY
3     ,
4     evento_nome VARCHAR2(200) NOT NULL,
5     evento_tipo VARCHAR2(50) NOT NULL,
6     impacto VARCHAR2(20) NOT NULL,
7     lugar_id NUMBER NOT NULL,
8     evento_data DATE NOT NULL
9
10    CONSTRAINT chk_impacto CHECK (impacto IN ('Bajo', 'Medio',
11        , 'Alto')),
12    CONSTRAINT uk_evento UNIQUE (evento_nome, lugar_id,
13        evento_data)
14
15    -- Chaves Foraneas
16    CONSTRAINT fk_evento_lugar FOREIGN KEY (lugar_id)
17        REFERENCES dim_lugar(lugar_id),
18    CONSTRAINT fk_evento_tempo FOREIGN KEY (evento_data)
19        REFERENCES dim_tempo(data),
20
21 );
22
23 -- Index en Oracle faise f ra do CREATE TABLE:
24 CREATE INDEX idx_evento_data_cidade ON dim_eventos(
25     evento_data, lugar_id);

```

2.3.2. Táboas de dimensión que cambian lentamente, implementadas con SCD tipo 2

```

1 CREATE TABLE dim_valorador (
2     valorador_chave NUMBER GENERATED ALWAYS AS IDENTITY
3     PRIMARY KEY, -- Chave surrogada
4     valorador_id NUMBER NOT NULL, -- ID do OLTP
5     idade NUMBER NOT NULL,
6     sexo CHAR(1) DEFAULT 'D' NOT NULL, -- H=Home, M=Muller, D
7     =Desco ecido O=Outro
8     data_nascimento DATE NOT NULL,
9     data_empezou_valer DATE NOT NULL,
10    data_deixou_valer DATE,
11    actual NUMBER(1) DEFAULT 1 NOT NULL, -- 1=TRUE, 0=FALSE
12    CONSTRAINT chk_sexo CHECK (sexo IN ('H', 'M', 'D', 'O'))
13 );

```

```

1 CREATE TABLE dim_hospedador (
2     hospedador_chave NUMBER GENERATED ALWAYS AS IDENTITY
3     PRIMARY KEY, -- Chave surrogada
4     hospedador_id NUMBER NOT NULL, -- ID do OLTP
5     tempo_resposta VARCHAR2(50) NOT NULL,
6     score_medio NUMBER(3,2),
7     numero_hospedaxes NUMBER NOT NULL,

```

```

7      e_superhost NUMBER(1) DEFAULT 0 NOT NULL, -- 1=TRUE, 0=
          FALSE
8      actual NUMBER(1) DEFAULT 1 NOT NULL -- 1=TRUE, 0=FALSE
9  );

```

2.3.3. Táboas ponte

```

1 CREATE TABLE ponte_evento_valoracion (
2     evento_id NUMBER NOT NULL,
3     valoracion_id NUMBER NOT NULL,
4     peso NUMBER(10,2) NOT NULL,
5     PRIMARY KEY (evento_id, valoracion_id),
6     FOREIGN KEY (evento_id) REFERENCES dim_eventos(evento_id)
7     ,
8     FOREIGN KEY (valoracion_id) REFERENCES feito_valoracion(
        valoracion_id)
9 );

```

```

1 CREATE TABLE ponte_hospedaxe_commodidade (
2     hospedaxe_id NUMBER NOT NULL,
3     commodidade VARCHAR2(50) NOT NULL,
4     peso NUMBER(10,2) NOT NULL,
5     PRIMARY KEY (hospedaxe_id, commodidade),
6     FOREIGN KEY (hospedaxe_id) REFERENCES dim_hospedaxe(
        hospedaxe_id)
7 );

```

```

1 CREATE TABLE ponte_hospedaxe_accesibilidade (
2     hospedaxe_id NUMBER NOT NULL,
3     accesibilidade VARCHAR2(50) NOT NULL,
4     peso NUMBER(10,2) NOT NULL,
5     PRIMARY KEY (hospedaxe_id, accesibilidade),
6     FOREIGN KEY (hospedaxe_id) REFERENCES dim_hospedaxe(
        hospedaxe_id)
7 );

```

2.3.4. Táboa de feitos

```

1 CREATE TABLE feito_valoracion (
2     valoracion_id NUMBER PRIMARY KEY, -- Do OLTP
3     hospedaxe_id NUMBER NOT NULL,
4     lugar_id NUMBER NOT NULL,
5     data DATE NOT NULL,
6     valorador_chave NUMBER NOT NULL, -- Chave surrogada (SCD
        Tipo 2)
7     hospedador_chave NUMBER NOT NULL, -- Chave surrogada (SCD
        Tipo 2)
8     score NUMBER(3,2) NOT NULL,

```

```

9      chuva_mm NUMBER(5,2) DEFAULT 0 NOT NULL, -- Dimensi n
        dexenerada
10
11      -- Chaves Foraneas
12      FOREIGN KEY (hospedaxe_id) REFERENCES dim_hospedaxe(
        hospedaxe_id),
13      FOREIGN KEY (lugar_id) REFERENCES dim_lugar(lugar_id),
14      FOREIGN KEY (data) REFERENCES dim_tempo(data),
15      FOREIGN KEY (valorador_chave) REFERENCES dim_valorador(
        valorador_chave),
16      FOREIGN KEY (hospedador_chave) REFERENCES dim_hospedador(
        hospedador_chave)
17 );

```

3. Diseño e implementación del ETL

- Diseñar e implementar el proceso ETL (Extract, Transform, Load) completo en Apache Hop, que permita extraer, depurar/limpiar, transformar y cargar los datos reales de Airbnb en el Data Warehouse creado en la tarea 2. El proceso deberá automatizar la carga desde los ficheros fuente hasta las tablas destino en Oracle, garantizando la calidad, consistencia y trazabilidad de los datos.
- Deberán especificarse los datasets de origen utilizados y documentar las principales transformaciones realizadas (limpieza, conversión de tipos, homogeneización de categorías, generación de claves sustitutas, etc.). Además, es necesario describir cómo se tratan valores nulos o inconsistentes, claves foráneas faltantes, formatos de fecha y moneda, posibles duplicados...
- El fichero principal de ejecución debe llamarse: `etl_airbnb_main.hpl` y debe ser ejecutable en Hop sin necesidad de editar las rutas internas. El entregable final debe incluir una carpeta datasets (o enlace a los ficheros fuente utilizados).
- El proyecto de Hop debe incluir al menos: Un job principal (.hpl) que coordine la ejecución completa del proceso ETL con varias transformaciones específicas para cada tabla. El pipeline debe ser ejecutable de forma completa desde un único job principal.
- Todas las rutas de ficheros y conexiones deben configurarse de manera relativa (no absolutas) para facilitar su ejecución en otro entorno. La conexión a Oracle debe configurarse mediante variables de entorno de Hop (`$ORACLE_HOST`, `$ORACLE_USER`, etc.), documentadas en el informe.
- Todas las transformaciones deben estar claramente nombradas y documentadas dentro del proyecto Hop. También deben utilizarse Hop variables para parametrizar rutas, nombres de archivos y credenciales, facilitando la reutilización.

- No es requisito mínimo, pero se valorará positivamente la inclusión de al menos una de las siguientes funcionalidades avanzadas: Gestión de dimensiones lentamente cambiantes (SCD tipo 1 o 2), control de carga incremental (detección de nuevos registros o cambios) o mecanismo de logging o control de errores mediante variables o salidas de Hop.
- *Conxunto de datos utilizados:*
- *Tratamento de nulos/inconsistentes:*
- *Claves foráneas*
- ...

4. Explotación del DW

- Se deben plantear y resolver tres consultas analíticas relevantes sobre el DW implementado. Para cada consulta, debe incluirse una breve descripción de la consulta (qué hace, y por qué es relevante o para qué podría aplicarse), y la sentencia SQL que resuelve dicha consulta sobre el esquema seleccionado. Las consultas deben de ser realistas y diferentes (utilizando diferentes tipos de consultas analíticas).
- Se debe crear un informe en Power BI que muestre visualizaciones interactivas de los datos, incluyendo alguna métrica DAX.

Consulta 1: Influe o número de hospedaxes dun hospedador sobre o seu score medio? Canto?

```

1 SELECT
2     h.numero_hospedaxes ,
3     COUNT(DISTINCT h.hospedador_chave) AS num_hospedadores ,
4     AVG(v.score) AS score_medio ,
5     COUNT(v.valoracion_id) AS total_valoraciones ,
6     -- Ratio de superhosts en este grupo
7     SUM(CASE WHEN h.e_superhost = 1 THEN 1 ELSE 0 END) *
8         100.0 /
9         COUNT(DISTINCT h.hospedador_chave) AS
10         porcentaje_superhosts
11 FROM feito_valoracion v
12 JOIN dim_hospedador h ON v.hospedador_chave = h.
13     hospedador_chave
14 WHERE h.actual = 1
15 GROUP BY h.numero_hospedaxes
16 HAVING COUNT(v.valoracion_id) >= 10 -- Filtrar grupos con
17     pocas valoraciones
18 ORDER BY h.numero_hospedaxes;
```


Consulta 2: Cales son os barrios con peores valoracións? Ordenados descendentemente pola valoración media da cidade.

```
1 WITH valoraciones_barrio AS (  
2     -- Calcular score medio por barrio  
3     SELECT  
4         l.lugar_id,  
5         l.barrio,  
6         l.cidade,  
7         l.pais,  
8         AVG(v.score) AS score_medio_barrio,  
9         COUNT(v.valoracion_id) AS num_valoraciones_barrio,  
10        STDDEV(v.score) AS desviacion_barrio  
11    FROM feito_valoracion v  
12    JOIN dim_lugar l ON v.lugar_id = l.lugar_id  
13    GROUP BY l.lugar_id, l.barrio, l.cidade, l.pais  
14 ),  
15 valoraciones_cidade AS (  
16     -- Calcular score medio por ciudad  
17     SELECT  
18         l.cidade,  
19         l.pais,  
20         AVG(v.score) AS score_medio_cidade,  
21         COUNT(v.valoracion_id) AS num_valoraciones_cidade  
22    FROM feito_valoracion v  
23    JOIN dim_lugar l ON v.lugar_id = l.lugar_id  
24    GROUP BY l.cidade, l.pais  
25 ),  
26 ranking_barrios AS (  
27     -- Combinar y rankear barrios dentro de cada ciudad  
28     SELECT  
29         vb.barrio,  
30         vb.cidade,  
31         vb.pais,  
32         vb.score_medio_barrio,  
33         vb.num_valoraciones_barrio,  
34         vc.score_medio_cidade,  
35         vc.num_valoraciones_cidade,  
36         -- Diferencia del barrio respecto a la media de su  
37         -- ciudad  
38         vb.score_medio_barrio - vc.score_medio_cidade AS  
39         diferencia_vs_cidade,  
40         -- Ranking dentro de la ciudad (1 = peor barrio)  
41         RANK() OVER (PARTITION BY vb.cidade, vb.pais  
42             ORDER BY vb.score_medio_barrio ASC) AS  
43             ranking_en_cidade  
44    FROM valoraciones_barrio vb  
45    JOIN valoraciones_cidade vc ON vb.cidade = vc.cidade AND  
46        vb.pais = vc.pais  
47    WHERE vb.num_valoraciones_barrio >= 5 -- Filtrar barrios  
48        con pocas valoraciones
```

```

44 )
45 SELECT
46     barrio,
47     cidade,
48     pais,
49     ROUND(score_medio_barrio, 2) AS score_barrio,
50     num_valoraciones_barrio,
51     ROUND(score_medio_cidade, 2) AS score_cidade,
52     ROUND(diferencia_vs_cidade, 2) AS diferencia,
53     ranking_en_cidade AS posicion_en_cidade
54 FROM ranking_barrios
55 WHERE ranking_en_cidade <= 5  -- Top 5 peores barrios por
    ciudad
56 ORDER BY
57     score_medio_cidade DESC,  -- Ciudades con mejor
    valoraci n primero
58     ranking_en_cidade ASC;    -- Dentro de cada ciudad,
    peores barrios primero

```

Consulta 3: Entre as comodidades e accesibilidades que pode ter un hospedaxe, cal beneficia máis á valoración ca súa presenza e cal perxudica máis ca súa ausencia?

```

1  -- Top 5 comodidades que M S BENEFICIAN (mayor score con
    ellas)
2  SELECT * FROM (
3      SELECT
4          comodidade,
5          'Comodidade' AS tipo,
6          ROUND(AVG(v.score), 2) AS score_medio
7      FROM feito_valoracion v
8      JOIN ponte_hospedaxe_comodidade pc ON v.hospedaxe_id =
        pc.hospedaxe_id
9      GROUP BY comodidade
10     HAVING COUNT(*) >= 10
11     ORDER BY AVG(v.score) DESC
12 ) WHERE ROWNUM <= 5
13
14 UNION ALL
15
16 -- Top 5 accesibilidades cuya AUSENCIA m s PERJUDICA (menor
    score sin ellas)
17 SELECT * FROM (
18     SELECT
19         accesibilidade AS comodidade,
20         'Accesibilidade_ (ausente)' AS tipo,
21         ROUND(AVG(v.score), 2) AS score_medio_sin_ella
22     FROM feito_valoracion v
23     WHERE NOT EXISTS (
24         SELECT 1

```

```
25         FROM ponte_hospedaxe_accesibilidade pa
26         WHERE pa.hospedaxe_id = v.hospedaxe_id
27     )
28     GROUP BY acessibilidade
29     HAVING COUNT(*) >= 10
30     ORDER BY AVG(v.score) ASC
31 ) WHERE ROWNUM <= 5;
```

Referencias

- [1] R. Kimball, “Snowflaked dimensions,” 2007, disponible en: <https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/dimensional-modeling-techniques/snowflake-dimension/>.
Accedido: 2025-11-10.
- [2] IBM, “Indexes for data warehouse applications,” 2025, disponible en: <https://www.ibm.com/docs/en/informix-servers/14.10.0?topic=indexes-data-warehouse-applications>.
Accedido: 2025-11-10.